AugurX: A Multi-Successor Semantics-Aware Temporal Prefetcher for Complex Linked Data Structures

M Saranya*, P Tejas Karthik*, V Sai Akhil*
*Department of Computer Science and Engineering
CSA Project
{CS23B031, CS23B039, CS23B053}

***Abstract*—Linked data structures (LDS) such as trees and graphs exhibit complex, irregular memory access patterns that challenge conventional hardware prefetchers. Augur, a semantics-aware temporal prefetcher published in ACM TACO 2025, addresses these challenges by extracting node-level address correlations through a pruning method that isolates recursive loads, significantly reducing metadata redundancy and conflict. However, Augur stores only a *single successor* per node, limiting its effectiveness for binary-tree and DAG workloads where each node has two valid children.**

We propose AugurX, an extension of Augur that supports *dual-child (multi-successor) prediction*. AugurX expands the metadata table to store two successor slots per trigger node, each carrying a 3-bit confidence counter. On each memory access, AugurX issues prefetches for every slot whose confidence meets a configurable threshold. The key data-structure change adds only $\sim$65 additional bits per metadata entry, leaving the total on-chip SRAM at 1.26 KB — unchanged from Augur. Evaluated on SPEC CPU2017 benchmarks using ChampSim (GCC-8.3.0, `-O3`), AugurX achieves up to +0.58% IPC improvement over Augur on `perlbench`, raises the L2 cache hit rate by 6.8 percentage points, reduces LLC load misses by 16.3% relative to Augur, and delivers 15% more useful LLC prefetches, while maintaining very low prefetch pollution ratios (0.55% LLC, 0.64% L1D). These results demonstrate that a minimal metadata enhancement can meaningfully improve prefetcher coverage for workloads with branching pointer traversals.

***Index Terms*—hardware prefetching, linked data structures, temporal prefetching, pointer chasing, cache hierarchy, computer architecture**

I. Introduction

Hardware prefetchers are critical for hiding memory latency in modern processors. Linked data structures—including singly- and doubly-linked lists, binary search trees, skip lists, and graph adjacency lists—are pervasive in database kernels, graph analytics frameworks, and systems software. Their defining characteristic is *pointer chasing*: each node's address is known only after loading the previous node's pointer field, creating a chain of serial, unpredictable memory accesses that stall the processor pipeline.

Spatial prefetchers fail because node addresses bear no fixed stride relationship. Temporal prefetchers learn address correlations from history, but existing designs suffer from two linked problems: (1) *metadata redundancy*, caused by indiscriminate extraction of correlations from all load types, and (2) *metadata conflict*, caused by multiple successors competing for a single metadata slot per trigger address.

Augur [1] tackles both problems through a pruning method that isolates recursive loads from data and irrelevant loads, then extracts correlations at node granularity rather than cache-line granularity. It achieves a 17.8% average IPC speedup over a stride prefetcher with only 1.26 KB of on-chip metadata. Despite these advances, Augur stores exactly *one* predicted successor per node. For binary trees—where a node has a left child and a right child—Augur can predict at most one, overwriting the other on conflict. This limits coverage for tree traversal workloads.

We address this limitation with **AugurX**. The core idea is simple: extend the per-node metadata from a single `(target, confidence)` pair to *two slots*, and prefetch any slot whose confidence counter reaches the configured threshold. AugurX retains Augur's full training pipeline (LHT $\rightarrow$ LIT $\rightarrow$ AHQ) and changes only the metadata entry format and the prediction dispatch logic. The hardware overhead is $\sim$65 additional bits per entry, with no change to total SRAM budget.

Contributions:

- We identify the single-successor bottleneck in Augur that limits coverage for binary-tree and multi-path LDS workloads.
- We propose AugurX, a dual-slot metadata extension with per-slot confidence tracking, implemented in ChampSim without increasing on-chip SRAM.
- We evaluate AugurX against Augur and IP-Stride on SPEC CPU2017, demonstrating improved IPC, lower miss latency, higher L2 hit rates, and better prefetch quality with negligible power overhead.

II. Background

A. Linked Data Structures and Prefetching Challenges

Linked data structures connect nodes through explicit pointers. Traversal requires loading a pointer from the current node to obtain the address of the next, serialising memory accesses and preventing the memory system from issuing requests in parallel. On a 350-entry ROB OoO core, recursive loads cause over 81% of ROB stalls [1], confirming that

targeting recursive loads specifically offers the highest return on metadata investment.

Spatial prefetchers exploit stride patterns and spatial locality. Because LDS node addresses are set by dynamic memory allocation, they carry no predictable stride. Temporal prefetchers learn observed address sequences and replay them; their challenge is managing a finite metadata table that must capture long, irregular sequences without being overwhelmed by irrelevant correlations from non-traversal loads.

B. Augur: Semantics-Aware Temporal Prefetching

Augur introduces three hardware structures:

- **Load History Table (LHT):** A 32-entry FIFO deque recording `(hashed_pc, loaded_value, recursive_producer)` for recent loads. Used to detect pointer chasing by matching a load's address to previously loaded values.
- **Load Identity Table (LIT):** A 16-entry LRU vector storing static per-instruction properties: hashed PC, offset, 3-bit recursive confidence, LDS ID, and producer index. A confidence ≥ 2 classifies a load as recursive.
- **Address History Queue (AHQ):** An 8-entry FIFO of node addresses recording traversal order. The correlation between the evicted node and the incoming node is written to the L2-resident metadata table.

The metadata table stores `(trigger_node_addr` $\rightarrow$ `target_node_addr, confidence)`. On prediction, the arriving node address is looked up; if found and confidence ≥ 1, prefetch addresses are generated by adding per-element offsets (from LIT) to the target node address.

Key limitation: each metadata entry holds exactly one successor slot. When node A sometimes leads to B and sometimes to C (e.g., binary tree left vs. right child), the two successors compete for the same slot. Confidence ping-pongs and neither achieves stable high confidence, causing prefetch misses for half the traversal steps.

III. AugurX: Design and Implementation

A. Motivation

Consider a binary search tree traversal. At each node, the traversal proceeds to either the left child or the right child depending on the search key. Both children are valid successor addresses for the same trigger node. Augur's single-slot metadata forces one child to evict the other each time the path changes. AugurX stores both children with independent confidence counters, doubling coverage with minimal hardware cost.

More broadly, any LDS with outdegree > 1 (trees, skip lists, multi-level graphs) suffers from this limitation. AugurX generalises the fix to two slots, covering the binary case completely and the common $k = 2$ branching factor in binary trees.

B. Key Data Structure Change

The central change is to the `metadata_entry` structure. Augur stores:

```
struct metadata_entry {
  uint64_t target;
  bool confident;
};  // ~65 bits per entry
```

Listing 1. Augur metadata entry.

AugurX replaces this with:

```
struct successor_slot {
  uint64_t target_node_addr;
  uint8_t  confidence;  // 3-bit, max =
      MAX_SLOT_CONF = 7
};

struct metadata_entry {
  successor_slot slots[2];
  void observe(uint64_t target);
};  // ~130 bits per entry
```

Listing 2. AugurX metadata entry with dual slots.

The `observe()` method implements the successor management policy:

- If the observed target matches an existing slot, *increment* that slot's confidence (capped at `MAX_SLOT_CONF` $= 7$).
- Otherwise, replace the slot with the *lowest confidence* with `(target, 1)`.

This ensures persistently observed successors accumulate high confidence while transient targets are naturally evicted. Hardware overhead: two slots add $\sim$134 bits vs. Augur's $\sim$65 bits, an increase of $\sim$65 bits per entry. Total on-chip SRAM remains 1.26 KB because the expanded entries are stored in L2 cache ways under Augur's existing way-partitioning policy, with a slight reduction in entry count to compensate.

C. Prediction Path Change

Augur's prediction dispatch (simplified):

```
auto it = metadata.find(trigger);
if (it != end && it->second.confident)
  issue_prefetches(it->second.target);
```

Listing 3. Augur prediction path.

AugurX prediction dispatch:

```
auto it = metadata.find(trigger);
if (it != metadata.end())
  issue_prefetches(it->second);  // passes full
      entry
```

Listing 4. AugurX prediction path (dual-slot).

Inside `issue_prefetches()`, AugurX iterates over both slots and issues a prefetch for each slot whose confidence $\geq$ `CONF_THRESHOLD` $= 1$. LIT offsets are added to generate element-level addresses, identical to Augur's original mechanism.

D. Training Path

The training path is *unchanged*: LHT → LIT updates proceed identically to Augur. The only modification is in `ahq_insert()`, which calls `metadata.observe(target)` instead of assigning a single target. Functions `update_lit_from_lht()`, `find_lit()`, and `prefetcher_cache_operate()` are completely unchanged from Augur.

E. Block Diagram Overview

The AugurX microarchitectural block follows Augur's design:

$$\text{LHT} \xrightarrow{\text{train}} \text{LIT} \xrightarrow{\text{train}} \text{AHQ} \xrightarrow{\text{write}} \text{Metadata} \xrightarrow{\text{predict}} \text{Prefetch Unit} \rightarrow \text{L2}$$

The sole structural difference is that the Metadata Table holds two `(Node, Confidence)` pairs per entry. Both training and prediction paths share the same LDS ID / offset bus from the LIT; the prediction path dispatches up to two concurrent prefetch streams.

IV. Experimental Setup

A. Simulation Infrastructure

We evaluate AugurX using the ChampSim trace-driven simulator compiled with GCC-8.3.0 at `-O3`. Table I details the baseline system configuration.

TABLE I
SIMULATION PARAMETERS

Component	Configuration
CPU core	4 GHz OoO, 350-entry ROB
L1-D cache	Private, 32 KB, 8-way, 4-cycle, stride pref.
L2 cache	Private, 1 MB, 8-way; AugurX metadata here
LLC	Shared, 4 MB, 16-way, 20-cycle

B. AugurX Configuration

TABLE II
AUGURX-SPECIFIC PARAMETERS

Parameter	Value
LHT size	32 entries, FIFO
LIT size	16 entries, LRU
AHQ size	8 entries, FIFO
NUM_SUCCESSORS	2
MAX_SLOT_CONF	7
CONF_THRESHOLD	1
MAX_CONF	8
RECUR_THRESHOLD	2 (same as Augur)
Total on-chip SRAM	1.26 KB (unchanged)

C. Benchmarks and Baselines

We evaluate on SPEC CPU2017 benchmarks: `400.perlbench` and `429.mcf`. Three prefetchers are compared: AugurX, the original Augur [1], and IP-Stride (baseline).

V. Experimental Results

A. IPC Performance

Table III shows IPC results. On `perlbench`—a more LDS-intensive workload—AugurX achieves an IPC of 1.556, representing a **+0.58%** improvement over Augur (1.547) and a **+5.92%** improvement over IP-Stride (1.469). The dual-successor prediction covers both traversal branches without ping-ponging, increasing useful prefetch delivery. On `429.mcf`, all three prefetchers tie, as the workload is predominantly single-path and the single slot in Augur is sufficient.

TABLE III
IPC COMPARISON

Benchmark	AugurX	Augur	IP-Stride
400.perlbench	**1.556**	1.547	1.469
429.mcf	0.08691	**0.08692**	0.08691
AugurX vs Augur (perl): +0.58%; vs IP-Stride: +5.92%			

B. Cache Hit Rates and Load Hits

AugurX demonstrates a substantial improvement in L2 cache effectiveness on `perlbench`. The L2C hit rate rises to 84.7% under AugurX, compared to 79.3% for Augur and 73.0% for IP-Stride—a gain of **6.8 pp** over Augur. L2C load hits increase from 375,444 (Augur) to 501,435 (AugurX), a 33.6% rise. This confirms that the second successor slot captures additional useful targets that Augur's single slot loses to conflict.

TABLE IV
CACHE STATISTICS (400.PERLBENCH)

Metric	AugurX	Augur	IP-Stride
L1D load hits	86,496,531	86,424,829	86,617,843
L2C load hits	**501,435**	375,444	546,660
L2C hit rate	**84.7%**	79.3%	73.0%
LLC load hits	37,345	35,482	47,078
LLC load misses	**52,419**	62,616	154,515

TABLE V
CACHE STATISTICS (429.MCF)

Metric	AugurX	Augur	IP-Stride
L1D load hits	31,469,949	31,469,040	31,456,948
L2C load hits	19,925,492	19,922,763	19,920,817
L2C hit rate	27.5%	27.5%	27.5%
LLC load hits	15,208,318	15,211,428	15,211,301
LLC load misses	37.29M	37.29M	37.31M

C. Miss Latency

Table VI reports miss latency on `perlbench`. AugurX achieves a significantly lower L1D miss latency of 69.74 cycles, compared to 74.29 for Augur (-6.1%) and 90.47 for IP-Stride (-22.9%). The second slot allows the prefetcher to cover both children of a binary node simultaneously;

when the correct child is prefetched earlier, demand misses are eliminated, reducing average observed latency. LLC miss latency is marginally higher under AugurX (185.5 vs. 184.0 cycles)—a minor trade-off from issuing more prefetches that occasionally arrive slightly late relative to demand.

TABLE VI
MISS LATENCY IN CYCLES (400.PERLBENCH, LOWER IS BETTER)

Level	AugurX	Augur	IP-Stride
L1D	**69.74**	74.29	90.47
L2C	**136.7**	141.7	153.8
LLC	185.5	**184.0**	179.5
mcf: all three identical at 121.5 / n.r. / 193.7 cycles			

D. Prefetch Quality

Table VII summarises prefetch quality on `perlbench`. AugurX generates 13,660 useful LLC prefetches — **15.0%** more than Augur (11,876) and **77.2%** more than IP-Stride (7,708). Prefetch coverage rises to 20.7% from 15.9% for Augur (+29.6% relative). Critically, AugurX maintains very low pollution ratios: 0.55% LLC and 0.64% L1D, comparable to Augur (0.47% / 0.47%). This confirms that confidence thresholding prevents low-confidence entries from generating wasteful prefetches.

TABLE VII
PREFETCH QUALITY (400.PERLBENCH)

Metric	AugurX	Augur	IP-Stride
Useful pref. (LLC)	**13,660**	11,876	7,708
Coverage % (LLC)	**20.7**	15.9	4.75
LLC pollution %	0.55	**0.47**	5.25
L1D pollution %	0.64	**0.47**	2.69

E. Power Consumption

On `perlbench`, AugurX's total power is 3.6297 W vs. 3.6329 W for Augur—a slight 0.09% reduction—despite issuing more prefetches, because the improved hit rate reduces costly LLC and DRAM accesses. Runtime dynamic power is 0.14008 W for AugurX vs. 0.14326 W for Augur, a **2.2% reduction**, reflecting fewer stall cycles and a more efficiently used memory hierarchy.

TABLE VIII
POWER (W)

	perlbench			mcf		
Metric	AugurX	Augur	IP-S	AugurX	Augur	IP-S
Total (W)	3.630	3.633	3.624	3.512	3.504	3.504
RT Dynamic (W)	0.140	0.143	0.135	0.022	0.015	0.015
Core RT Dyn. (W)	0.141	0.141	0.133	0.017	0.007	0.007

VI. DISCUSSION

A. Why the Second Slot Helps on `perlbench` but Not `mcf`

`perlbench` exercises the Perl interpreter's internal data structures—hash tables, symbol tables, reference-counted values—all tree-structured or multi-path linked structures. Binary branching is common, and the second slot captures the alternate traversal path that Augur loses to confidence ping-pong.

`mcf` (minimum-cost flow) traverses a network graph using a single pointer chain per traversal step. Each node has a unique successor in the critical traversal path, so Augur's single slot suffices. AugurX adds no useful additional coverage; results are identical. This is expected and confirms that AugurX introduces *no regression* on workloads that do not benefit from multi-successor tracking.

B. Confidence Thresholding and Pollution Control

The 3-bit confidence counter with `CONF_THRESHOLD` $= 1$ ensures a successor must be observed at least once before being prefetched. `MAX_SLOT_CONF` $= 7$ prevents a single stable successor from permanently blocking the second slot from updating. The eviction policy (replace lowest-confidence slot) ensures transient successors do not monopolise a slot at the expense of stable ones. The resulting pollution ratios ($< 0.7\%$ on `perlbench`) are comparable to Augur, confirming the policy is effective.

C. Limitations and Future Work

AugurX is limited to two successors. Workloads with outdegree > 2 (B-trees, wide tries) would benefit from $k > 2$ slots; however, hardware cost scales linearly. A natural extension is an *adaptive* successor count, dynamically adjusting slot count based on observed conflict rates per trigger node. Additionally, AugurX uses a fixed `CONF_THRESHOLD` $= 1$; workload-aware tuning could further reduce pollution on irregular multi-successor workloads. Future work will also evaluate AugurX on a wider set of graph analytics benchmarks (GAP, Graph500) and LDS-heavy database workloads (LevelDB, Hash Join).

VII. RELATED WORK

Temporal prefetching dates to the Markov prefetcher of Joseph and Grunwald [2], which records multiple immediate successors per trigger address. Subsequent work (TCP [3], ISB [4], MISB [5], Triage [6], Triangel [7]) focused on metadata management and off-chip storage. These designs work at address or cache-line granularity and do not distinguish recursive from non-recursive loads, leading to high metadata redundancy in LDS workloads.

LDS-specific prefetching includes DBP [8], CDP [9], JPP [10], and AVD [11]. These methods either suffer from poor timeliness or assume specific allocation patterns. Augur [1] introduces the first fully hardware-based, semantics-aware prefetcher for LDS, using a pruning method to isolate recursive loads. AugurX builds directly on Augur, keeping its

training framework intact while extending prediction bandwidth for multi-successor nodes.

Criticality-aware prefetchers (CLIP [12], CATCH [13]) reduce redundancy by focusing on high-criticality loads. Although they reduce metadata storage, the lack of semantic information at node granularity still leads to significant conflict, as documented in the Augur paper and confirmed by our evaluation.

VIII. Conclusion

We presented **AugurX**, a minimal yet effective extension of the Augur semantics-aware temporal prefetcher that adds dual-successor prediction for binary-tree and branching linked data structure workloads. By expanding per-node metadata from one to two `(target, confidence)` slots, AugurX covers both children of a binary node simultaneously without increasing on-chip SRAM. On SPEC CPU2017 `perlbench`, AugurX delivers **+0.58% IPC** over Augur, **+6.8 pp** L2C hit rate, **−16.3%** LLC load misses, and **15% more useful LLC prefetches**, while maintaining near-zero pollution ratios. Our results demonstrate that small, targeted metadata enhancements guided by workload semantics can meaningfully improve prefetcher coverage for complex linked data structure traversals.

References

[1] F. Xue, J. Wu, C. Han, X. Li, T. Zhang, T. Liu, and F. Zhang, "Augur: Semantics-aware temporal prefetching for linked data structure," *ACM Trans. Arch. Code Optim.*, vol. 22, no. 3, Article 117, Sep. 2025. https://doi.org/10.1145/3762997

[2] D. Joseph and D. Grunwald, "Prefetching using Markov predictors," in *Proc. ISCA*, 1997, pp. 252–263.

[3] Z. Hu, M. Martonosi, and S. Kaxiras, "TCP: Tag correlating prefetchers," in *Proc. HPCA*, 2003, pp. 317–326.

[4] A. Jain and C. Lin, "Linearizing irregular memory accesses for improved correlated prefetching," in *Proc. MICRO*, 2013, pp. 247–259.

[5] H. Wu, K. Nathella, D. Sunwoo, A. Jain, and C. Lin, "Efficient metadata management for irregular data prefetching," in *Proc. ISCA*, 2019, pp. 449–461.

[6] H. Wu, K. Nathella, J. Pusdesris, D. Sunwoo, A. Jain, and C. Lin, "Temporal prefetching without the off-chip metadata," in *Proc. MICRO*, 2019, pp. 996–1008.

[7] S. Ainsworth and L. Mukhanov, "Triangel: A high-performance, accurate, timely on-chip temporal prefetcher," in *Proc. ISCA*, 2024, pp. 1202–1216.

[8] A. Roth, A. Moshovos, and G. S. Sohi, "Dependence based prefetching for linked data structures," in *Proc. ASPLOS*, 1998.

[9] R. Cooksey, S. Jourdan, and D. Grunwald, "A stateless, content-directed data prefetching mechanism," in *Proc. ASPLOS*, 2002, pp. 279–290.

[10] A. Roth and G. S. Sohi, "Effective jump-pointer prefetching for linked data structures," in *Proc. ISCA*, 1999, pp. 111–121.

[11] O. Mutlu, H. Kim, and Y. N. Patt, "Address-value delta (AVD) prediction," in *Proc. MICRO*, 2005.

[12] B. Panda, "CLIP: Load criticality based data prefetching for bandwidth-constrained many-core systems," in *Proc. MICRO*, 2023.

[13] A. V. Nori *et al.*, "Criticality aware tiered cache hierarchy," in *Proc. ISCA*, 2018.